

Lecture 2

Supplement 3: decision tree algorithm, ensemble, and random forests

**01211373 : Machine Learning and
Programming for Industry**

Dr. Varodom Toochinda

Dept. of Mechanical Engineering,
Kasetsart University



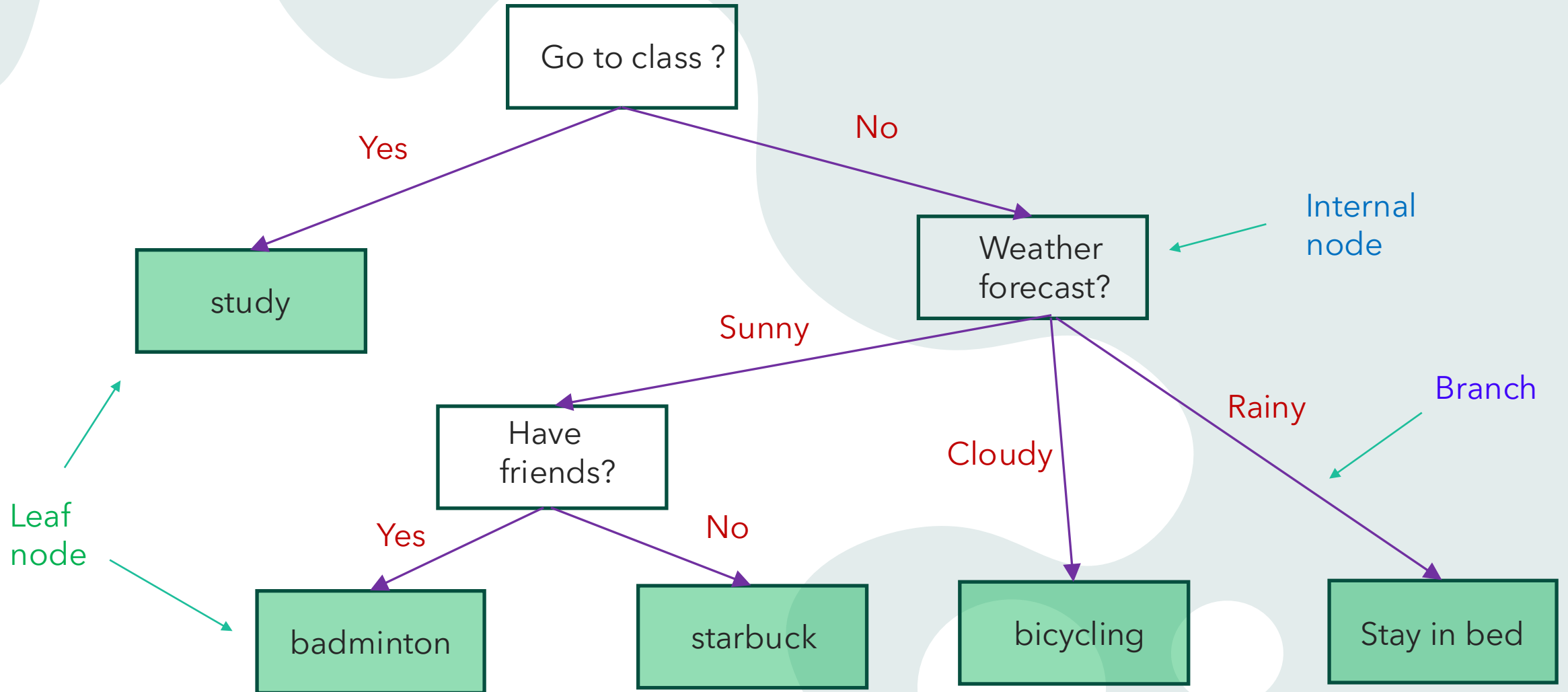
Topics

- ❑ Decision Trees
- ❑ Ensemble Methods
 - ❑ Majority voting
 - ❑ Bagging
- ❑ Random Forests

AI overview of decision tree

"A decision tree is a flowchart-like model that helps in making decisions by mapping out a sequence of choices, their potential outcomes, costs, benefits, and probabilities. It consists of nodes and branches, with decision nodes representing choices, chance nodes indicating uncertain outcomes, and leaf nodes signifying the final result. Decision trees are used in various fields, from machine learning to business and medicine, to classify data, predict outcomes, and guide complex decision-making processes in an easy-to-understand, hierarchical structure."

Decision tree example



Generic (Binary) Tree Growing Algorithm

1. Pick the feature that, when parent node is split, results in largest information gain
2. Stop if child nodes are pure, or information gain ≤ 0
3. Go back to step 1 for each of the two child nodes

Decision Tree in Pseudocode (recursive)

GenerateTree($\mathcal{D}$):

if $y = 1 \ \forall (X,y) \in \mathcal{D}$ or $y = 0 \ \forall (X,y) \in \mathcal{D}$:

return Tree

else:

pick best feature x_j :

$\mathcal{D}_0$ at Child₀ : $x_j = 0 \ \forall (X,y) \in \mathcal{D}$

$\mathcal{D}_1$ at Child₁ : $x_j = 1 \ \forall (X,y) \in \mathcal{D}$

return Node(x_j , GenerateTree($\mathcal{D}_0$), GenerateTree($\mathcal{D}_1$))

Design Choices

- ❑ How to split
 - ❑ what measurement/criterion as measure of goodness
 - ❑ binary v.s. multi-category split
- ❑ When to stop
 - ❑ if leaf nodes contain only examples of the same class
 - ❑ feature values are all the same for all examples
 - ❑ statistical significance test

Types of Decision Tree

❑ ID3 (Iterative Dichotomizer 3)

- ❑ discrete features, binary and multi-category
- ❑ short and wide trees, no pruning, prone to overfitting
- ❑ cannot handle numeric features
- ❑ maximizing information gain/minimizing entropy

❑ C4.5

- ❑ continuous and discrete features. Continuous is very expensive
- ❑ post-pruning

❑ CART (Classification And Regression Trees)

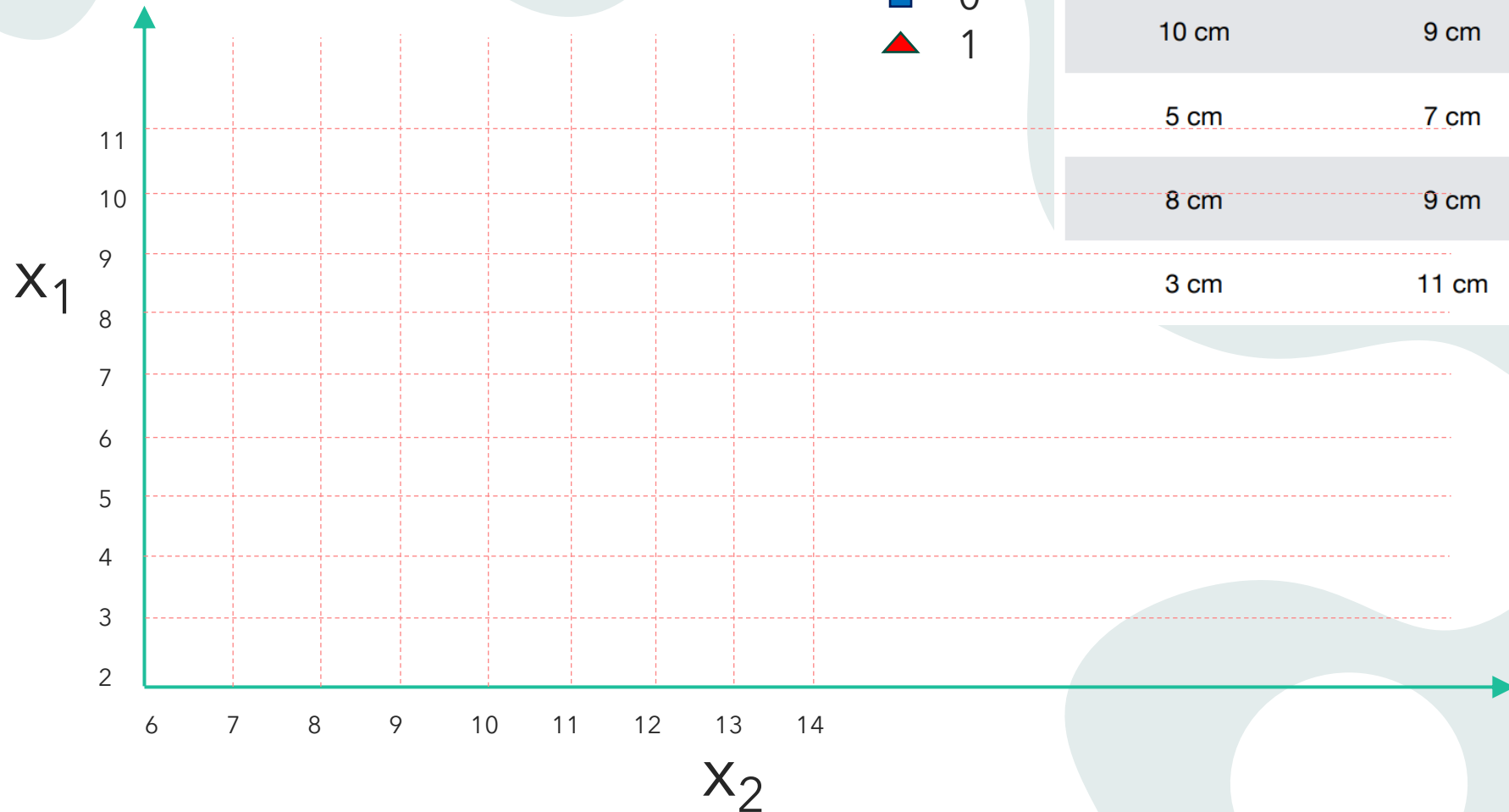
- ❑ continuous and discrete features
- ❑ strictly binary splits (taller trees)
- ❑ variance reduction in regression trees
- ❑ Gini impurity measure
- ❑ cost complexity pruning

❑ Others

- ❑ CHAID (Chi-squared Automatic Interaction Detector)
- ❑ MARS (Multivariate Adaptive Regression Splines)
- ❑ C5.0

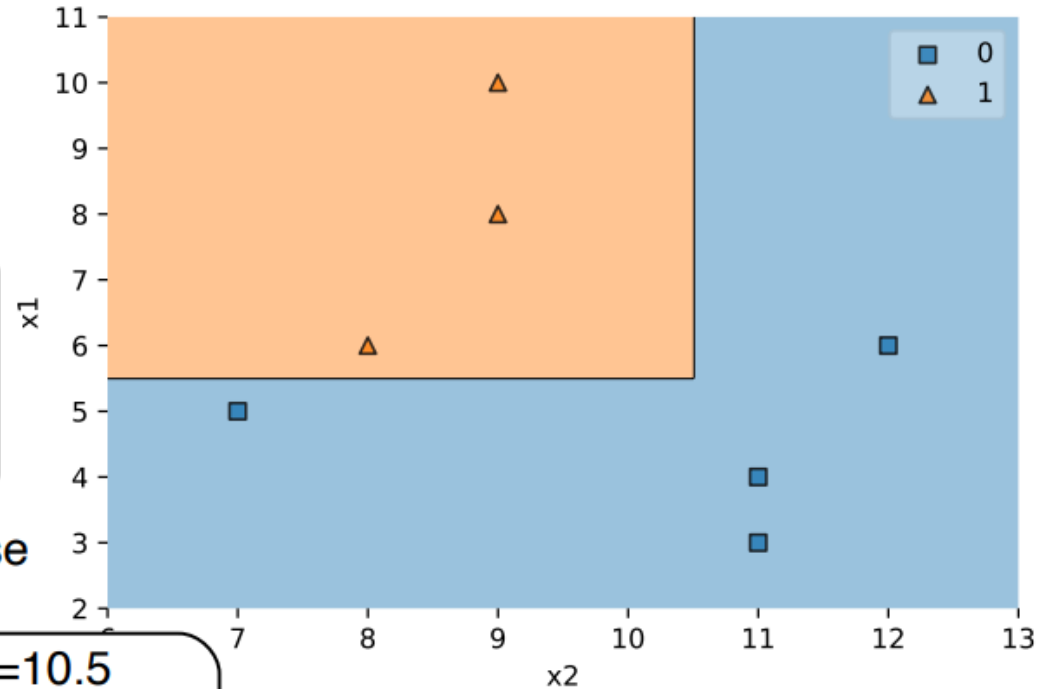
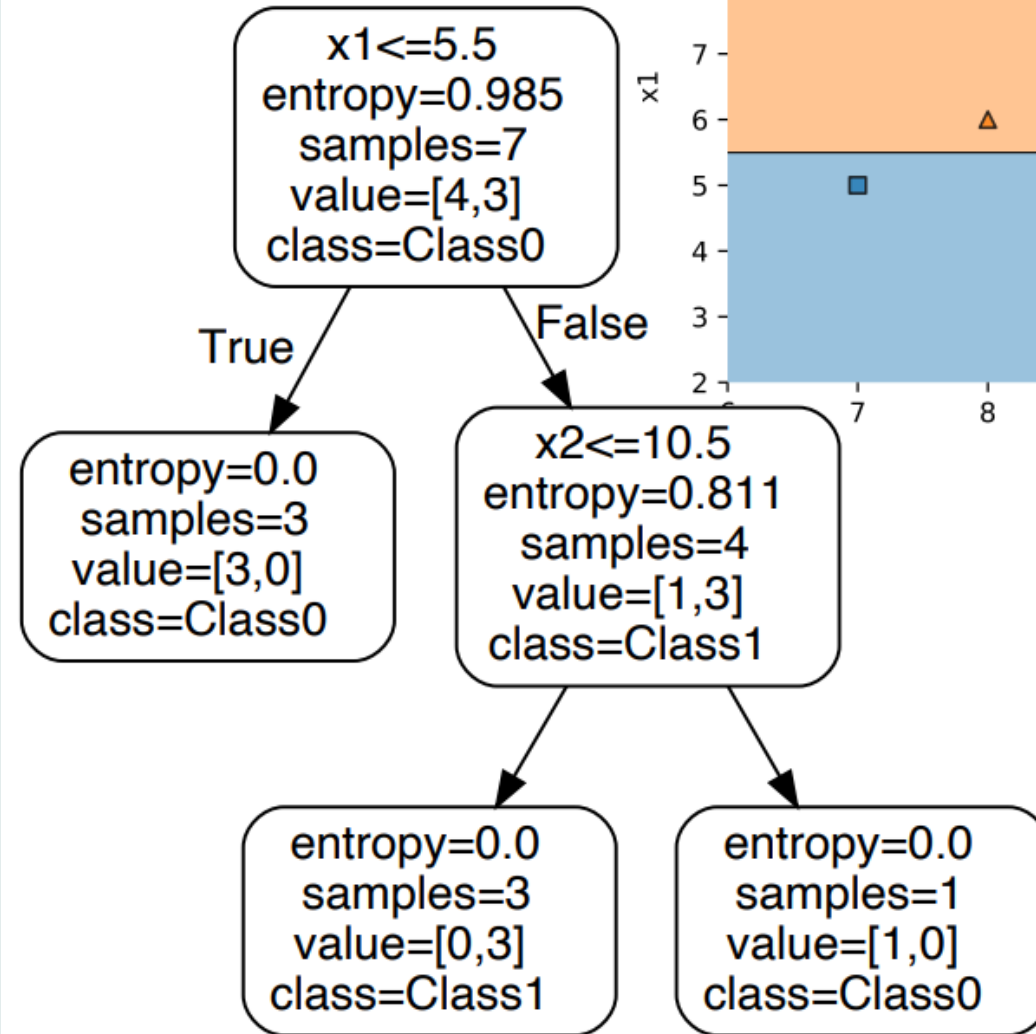
Example : drawing a decision boundary

■ 0
▲ 1

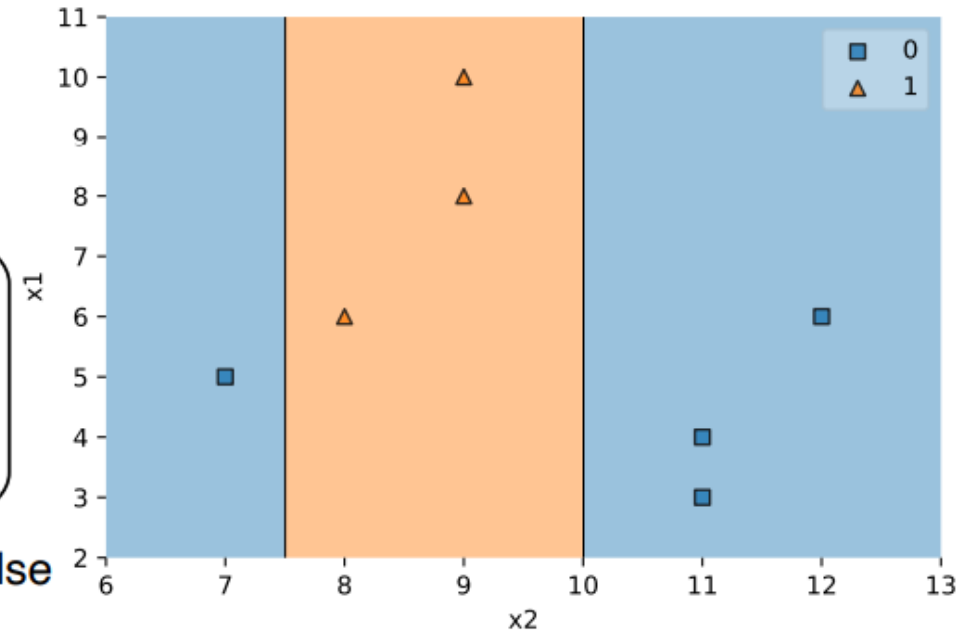
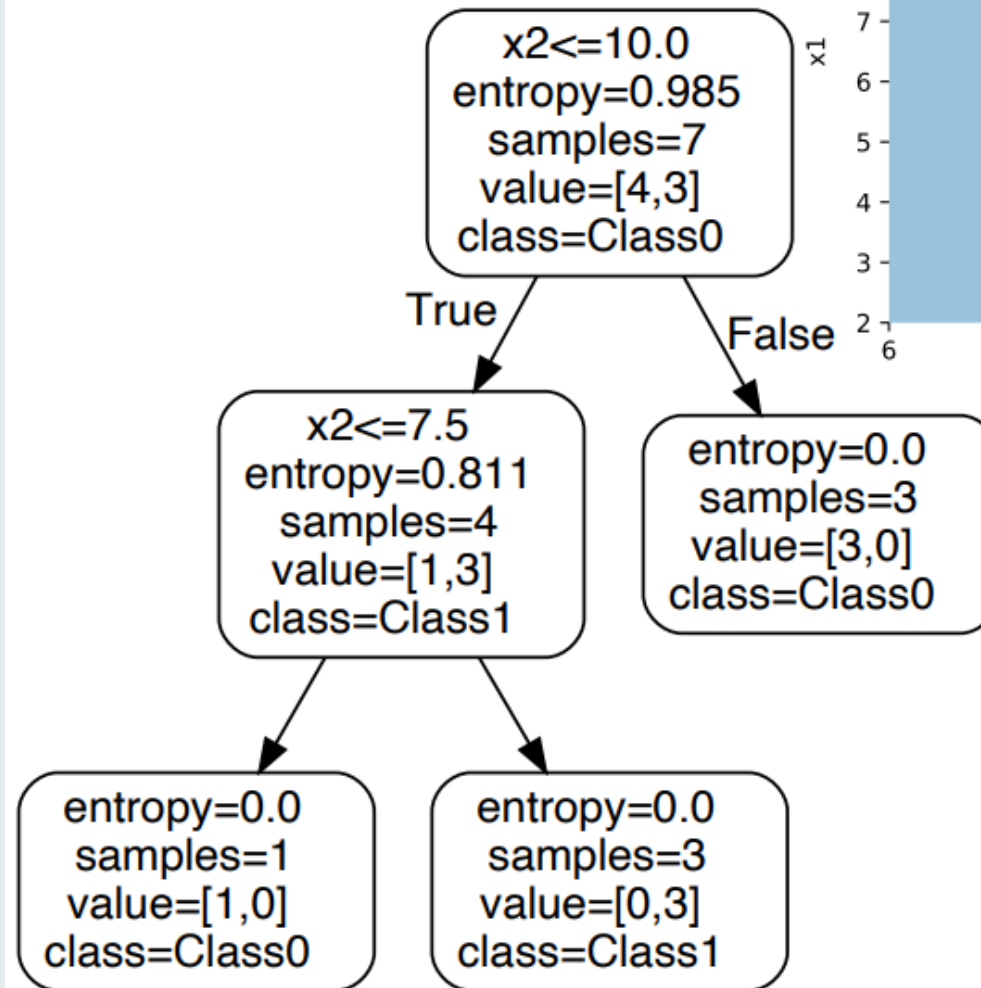


x_1	x_2	x_3	y
6 cm	8 cm	9 cm	1 ▲
4 cm	11 cm	2 cm	0 ■
6 cm	12 cm	4 cm	0 ■
10 cm	9 cm	3 cm	1 ▲
5 cm	7 cm	8 cm	0 ■
8 cm	9 cm	3 cm	1 ▲
3 cm	11 cm	5 cm	0 ■

Decision Tree # 1



Decision Tree # 2



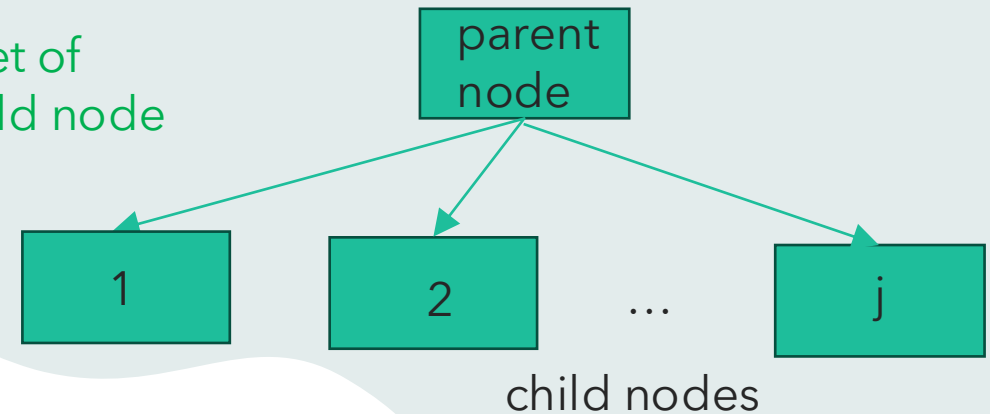
Information Gain (IG)

General case

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$

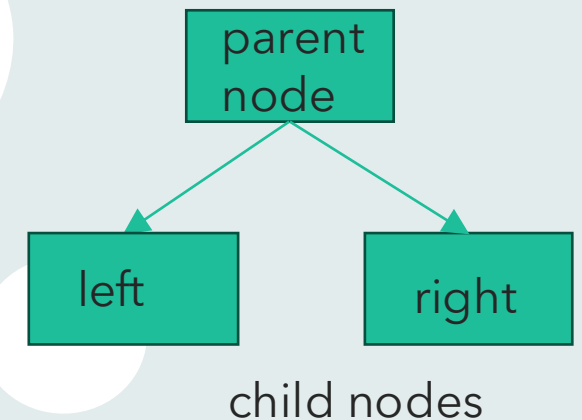
Annotations for the general case formula:

- $I(D_p)$: impurity measures (purple arrow)
- D_p : dataset of parent node (blue arrow)
- N_j : total number of examples in jth child node (red arrow)
- N_p : total number of examples in parent node (teal arrow)
- $I(D_j)$: impurity measures (purple arrow)
- D_j : dataset of jth child node (green arrow)



Binary DT

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$



Impurity measures for binary DT

Entropy

$$I_H(t) = - \sum_{i=1}^c p(i|t) \log_2(i|t)$$

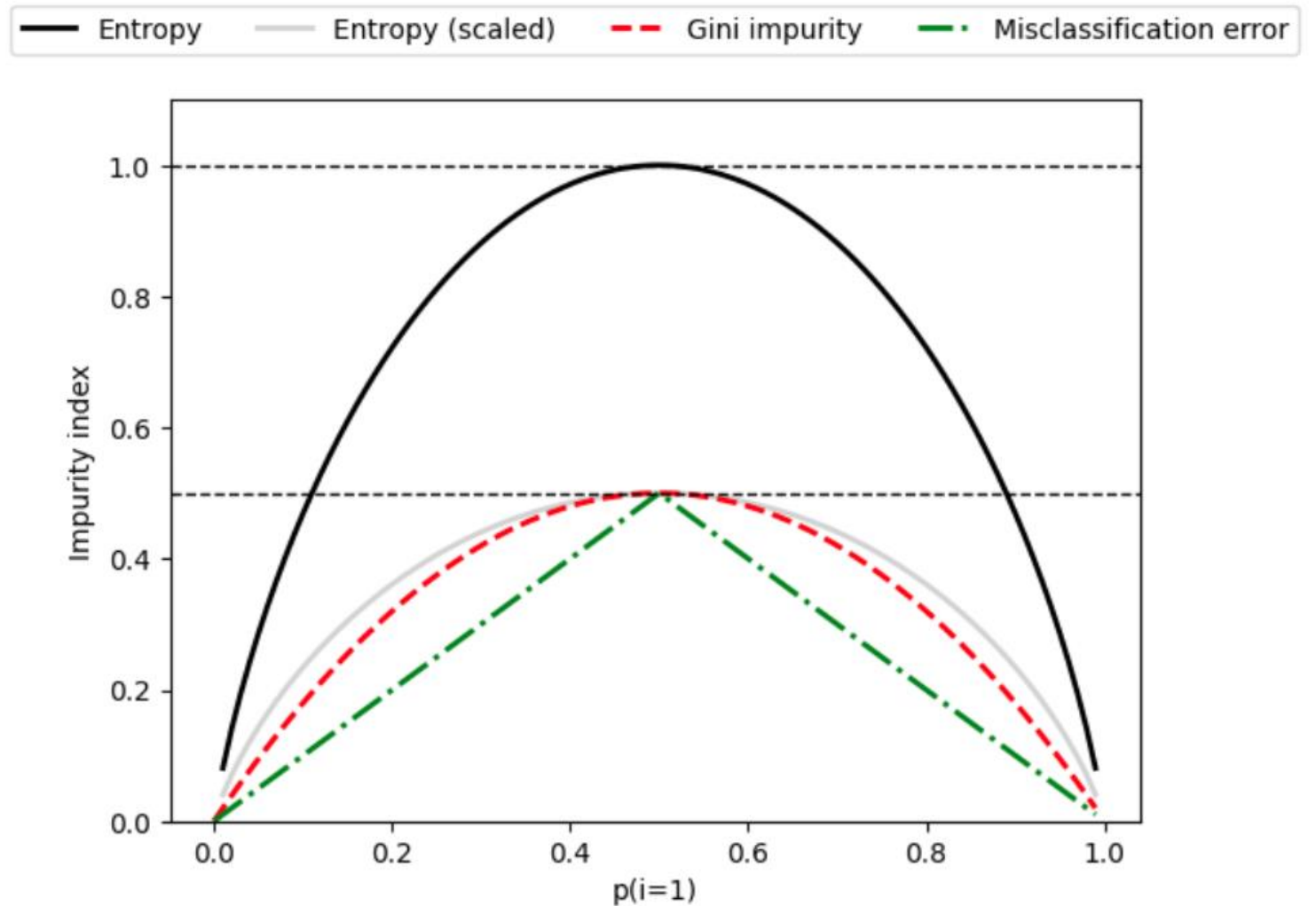
Gini

$$I_G(t) = \sum_{i=1}^c p(i|t)(1 - p(i|t)) = 1 - \sum_{i=1}^c p(i|t)^2$$

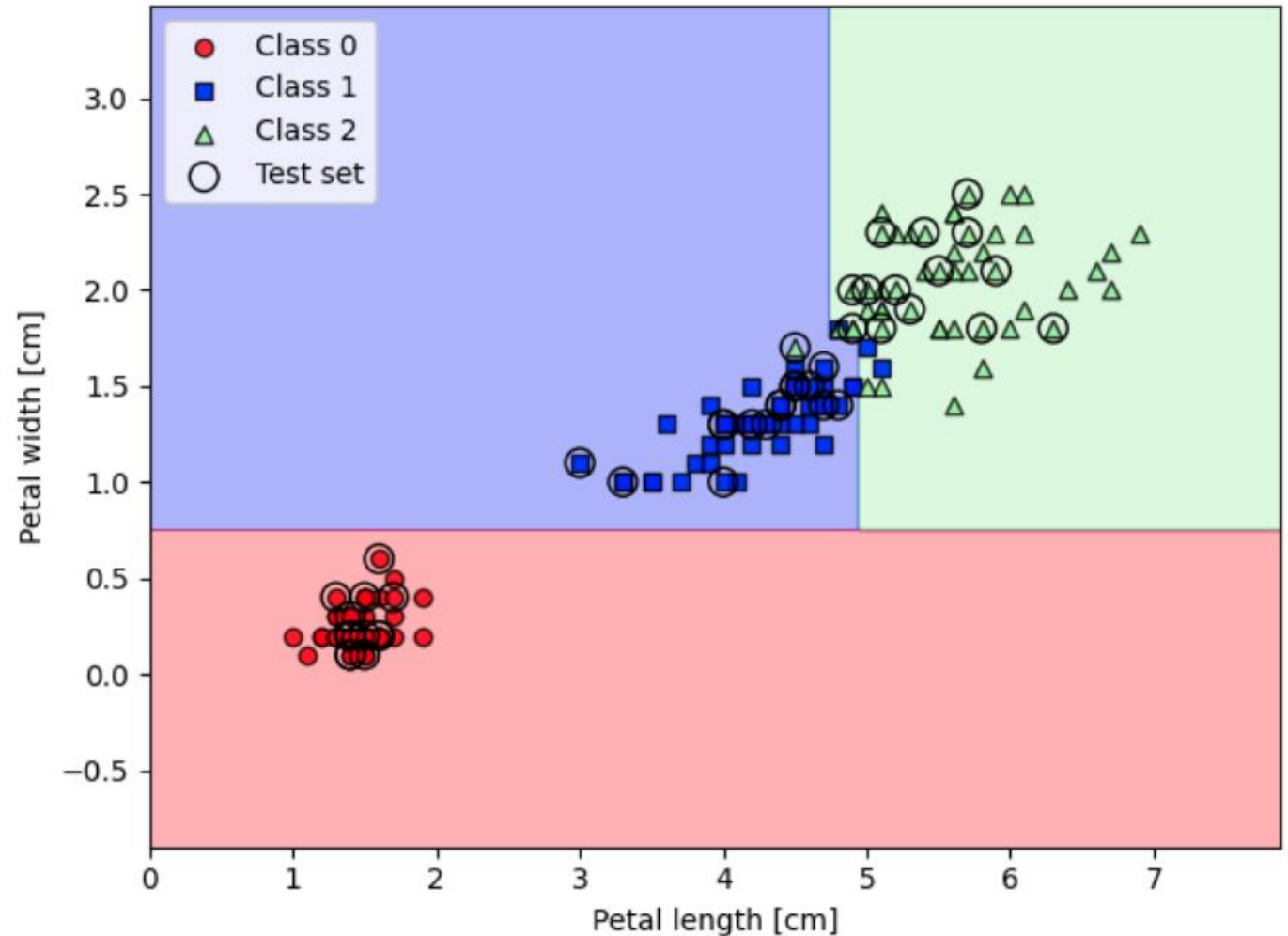
Classification error

$$I_E(t) = 1 - \max p(i|t)$$

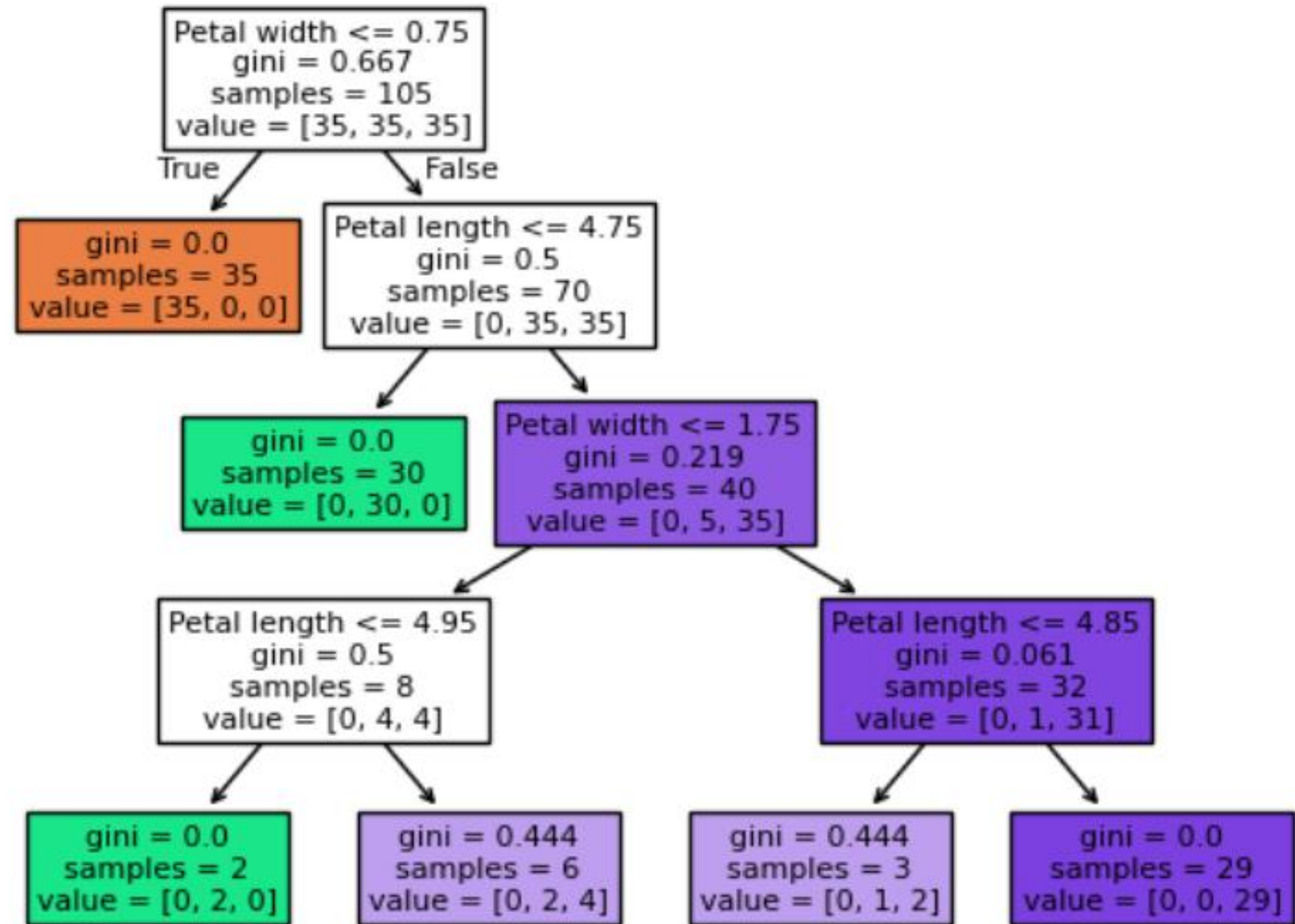
impurity criteria comparison



DT classification on iris dataset



plot of decision tree on iris dataset





Decision trees

- Pros
 - Interpretability (flowchart-like, human readable)
 - Can naturally handle both numerical and categorical data
 - No feature scaling required
 - Can model nonlinear relationships
- Cons
 - Prone to overfitting
 - Less effective for large/complex problem (compared to NN)
 - Provide discrete, step-like prediction

Ensemble methods

Ensemble approach based on majority voting

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A(C_j(x) = i)$$

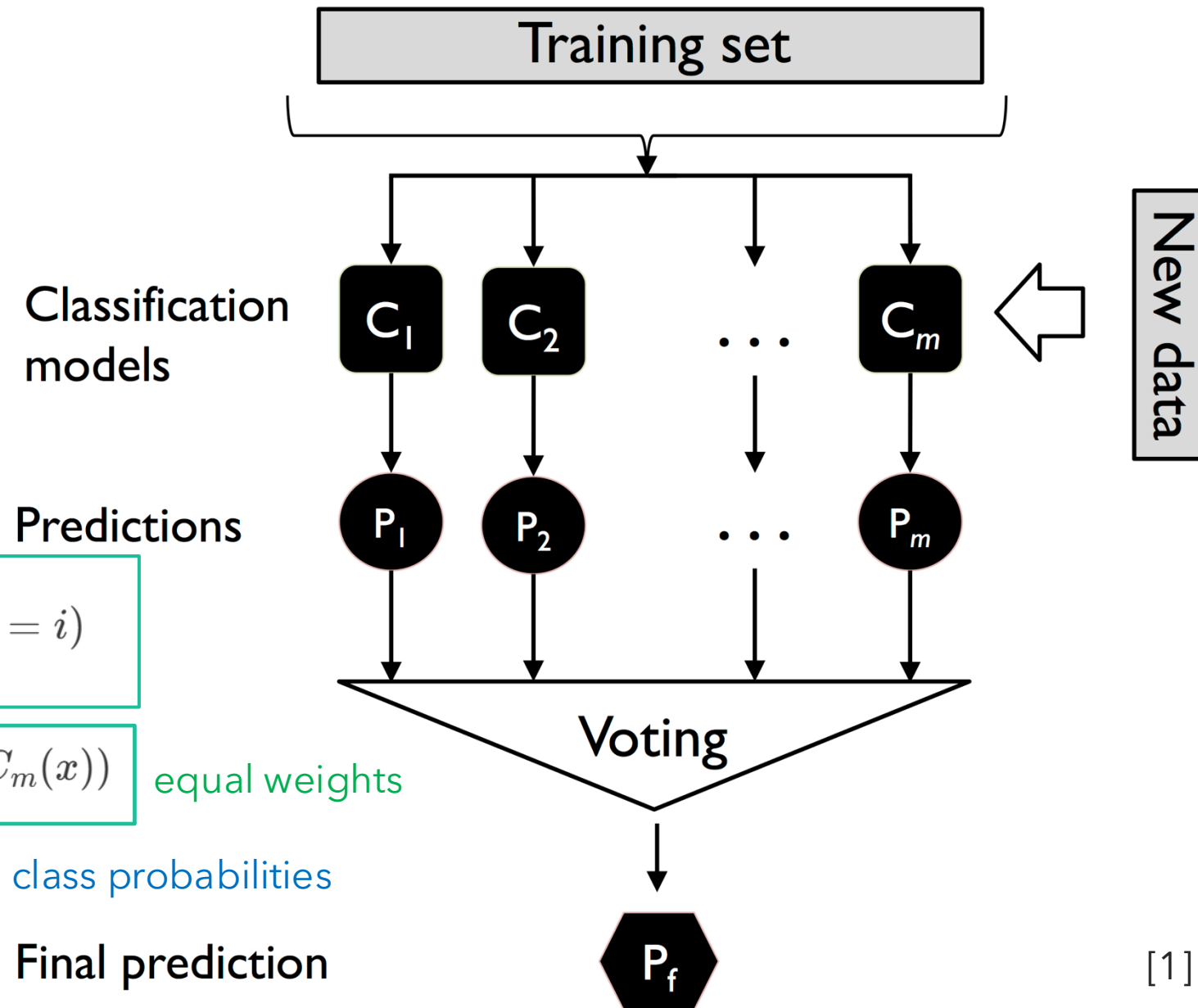
$$\hat{y} = \text{mode}(C_1(x), C_2(x), \dots, C_m(x))$$

equal weights

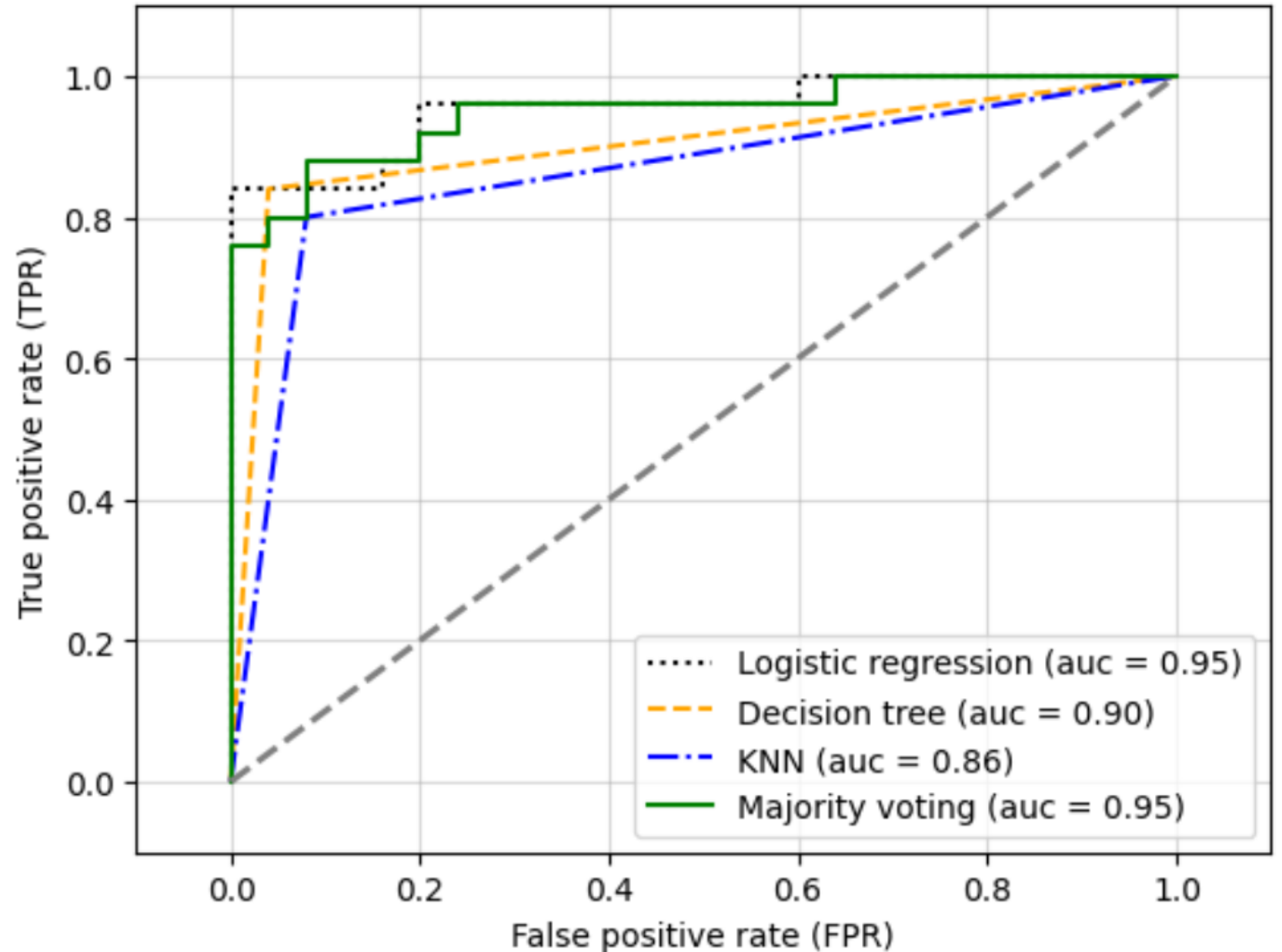
$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j p_{ij}$$

class probabilities

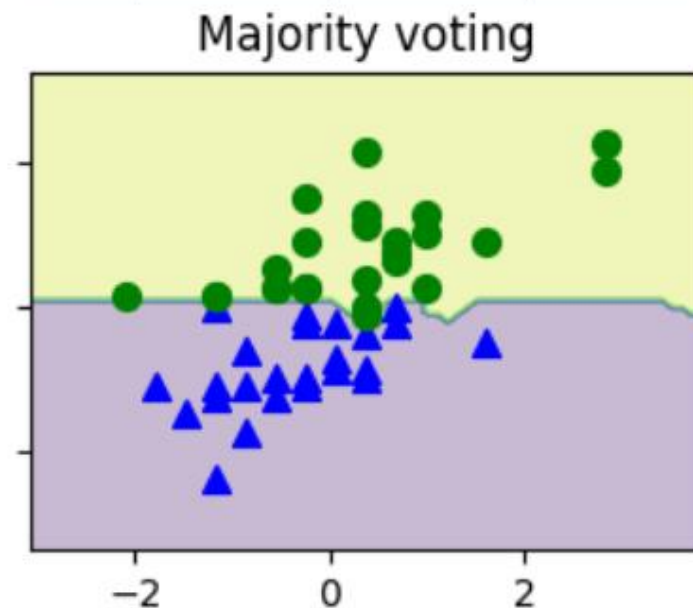
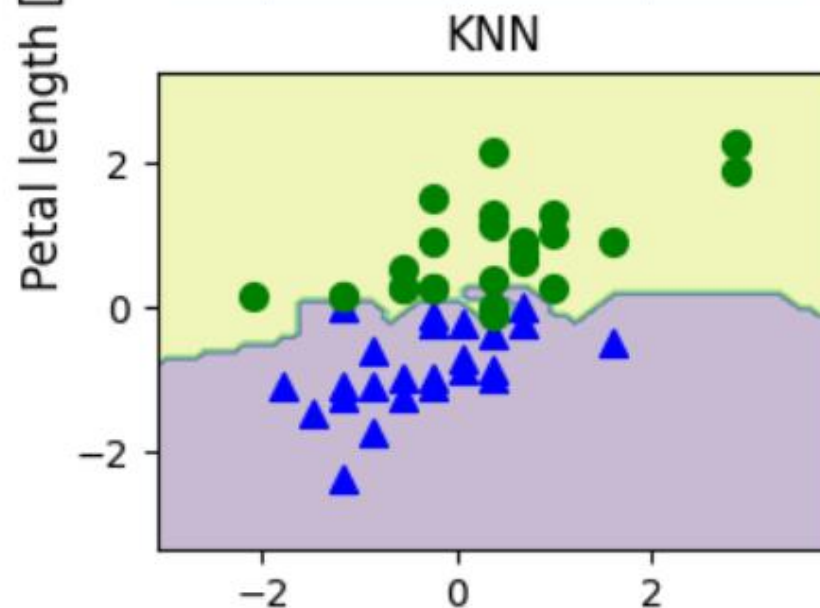
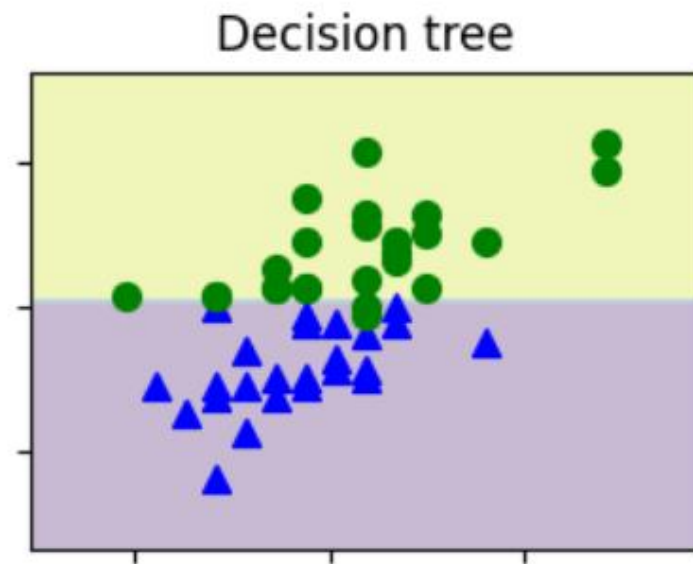
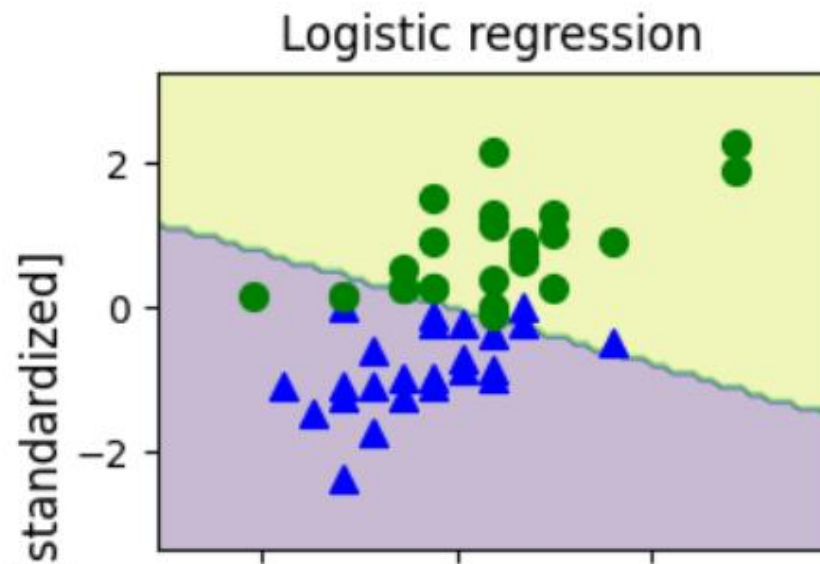
Final prediction



Evaluating and tuning ensemble classifier

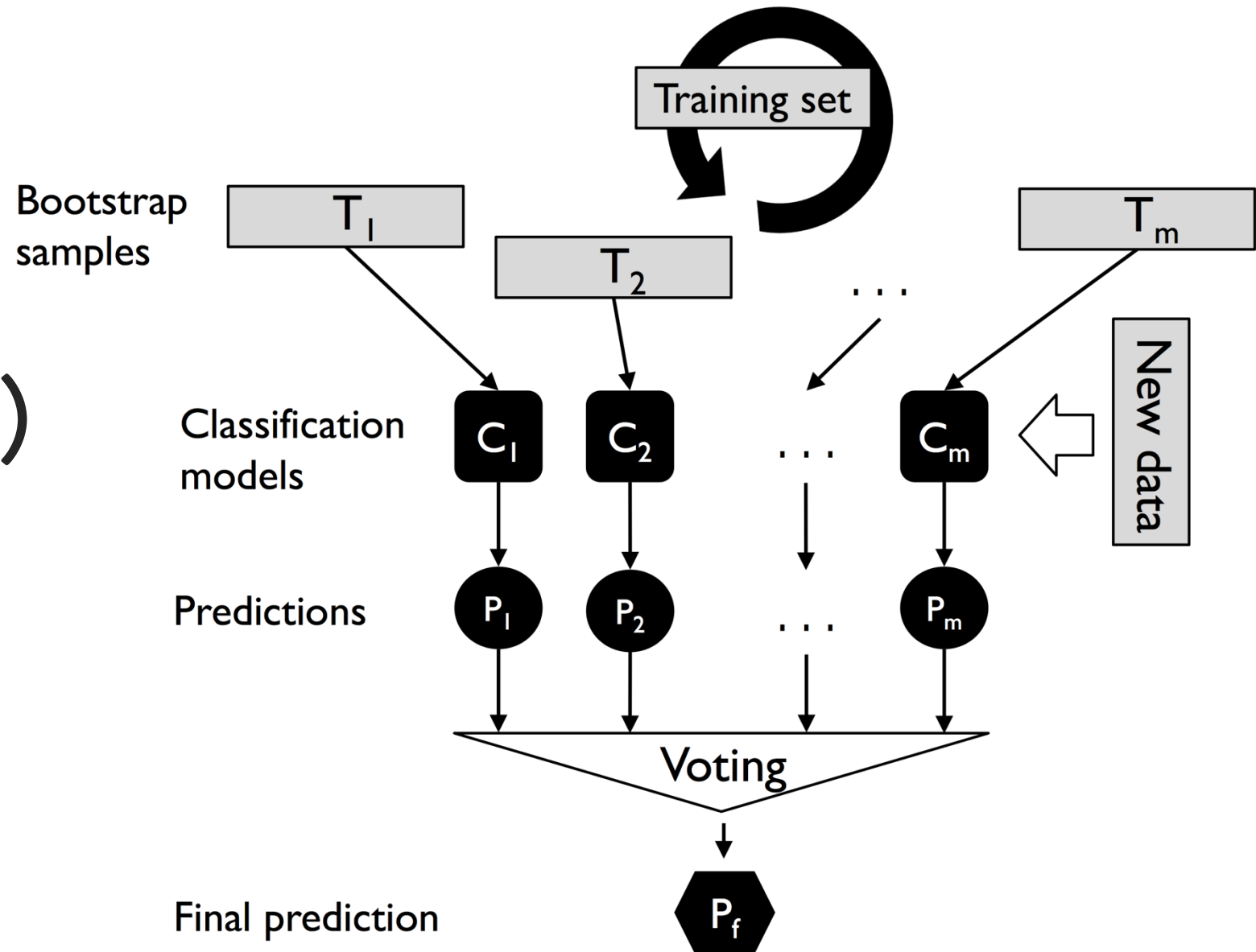


Decision boundaries for the different classifiers



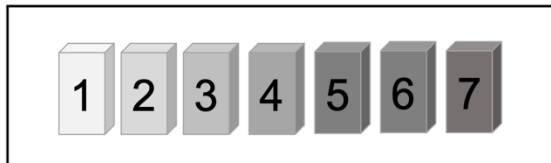
Sepal width [standardized]

Bagging (bootstrap aggregating)



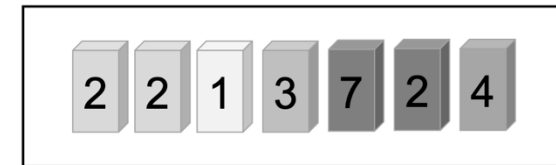
Bagging example

Training set
with 7 training examples:

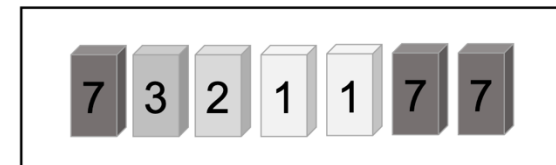


Sampling with
replacement

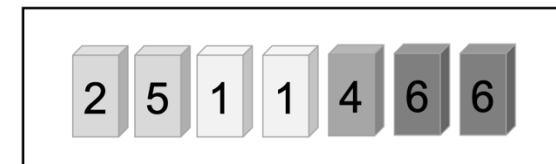
Bagging round 1
Bagging round 1
...
Bagging round m



Classifier
 C_1

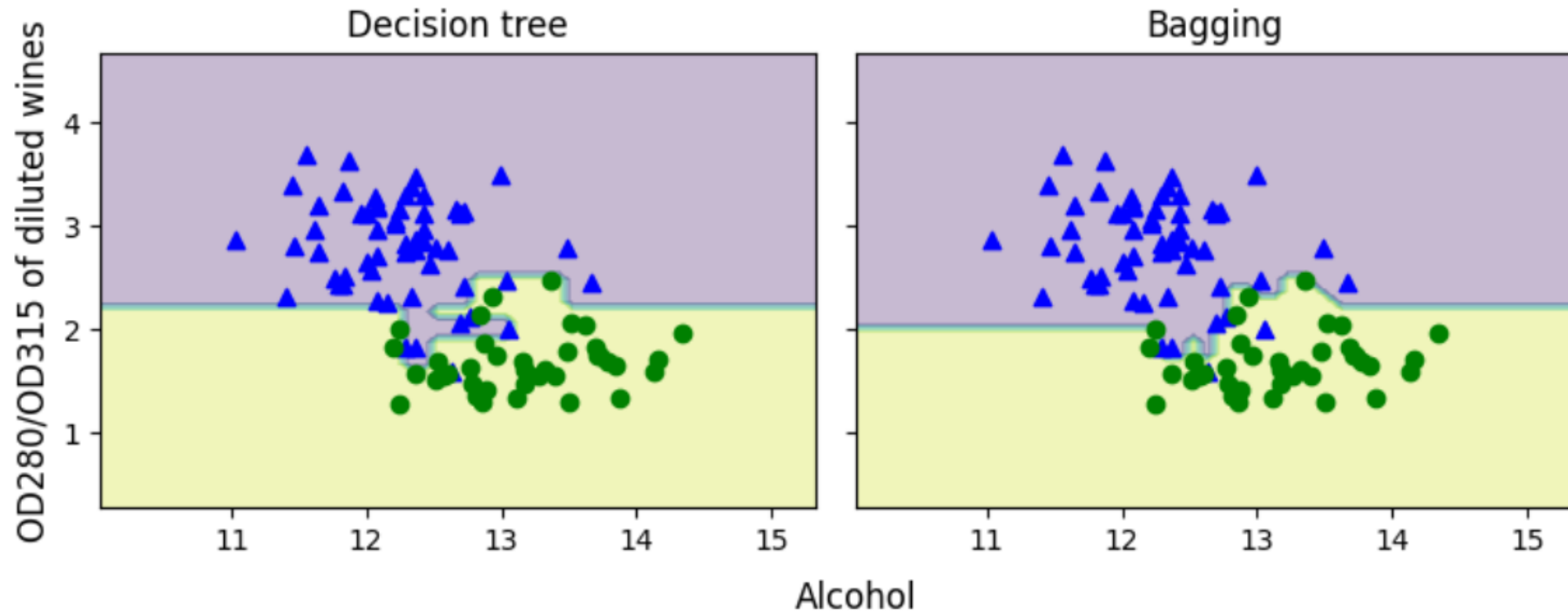


Classifier
 C_2



Classifier
 C_m

Decision regions for wine dataset



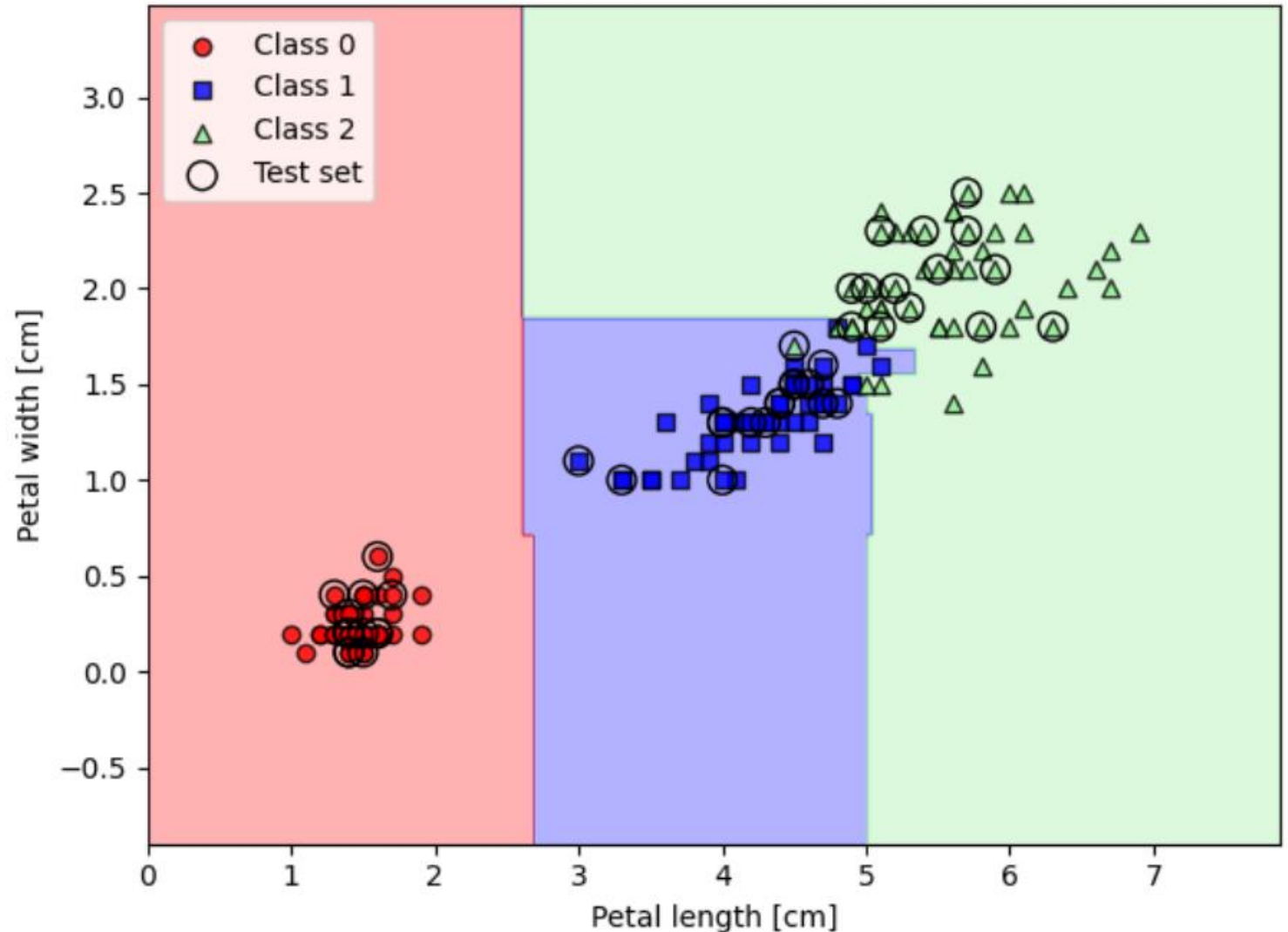
A photograph of a dense forest with tall, slender trees and a thick canopy of green leaves. Sunlight filters through the canopy, creating a dappled light effect on the forest floor. The text "Random forests" is overlaid in white at the bottom left.

Random forests

Random forest algorithm

1. Draw a random **bootstrap** sample of size n (randomly choose n examples from the training dataset with replacement).
2. Grow a decision tree from the bootstrap sample. At each node:
 - a. Randomly select d features without replacement.
 - b. Split the node using the feature that provides the best split according to the objective function, for instance, maximizing the information gain.
3. Repeat *steps 1-2* k times.
4. Aggregate the prediction by each tree to assign the class label by **majority vote**

Classification of iris dataset with random forest



References



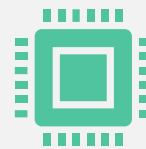
1. S. Raschka, Y. Liu and V. Mirjalili. Machine Learning with PyTorch and Scikit-Learn. Packt Publishing. 2022.

<https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>



2. A. Ng. CS229 Lecture Notes. Stanford University. 2023.

https://cs229.stanford.edu/main_notes.pdf



3. T. Broderick. Introduction to Machine Learning course (6.036). Massachusetts Institute of Technology. 2020.

<https://tamarabroderick.com/ml.html>

4. S. Raschka. **Introduction to Machine Learning course**. University of Wisconsin, Madison. 2021